

Assignment 1: Stack Machine Compiler

IC526 Advanced Compilers
Fall 2026

Objective

Implement a compiler from a small arithmetic-expression language to a stack machine, together with interpreters for the target language. The assignment practices algebraic data types, recursion, lists, pattern matching, options, and exceptions in OCaml.

1. Languages

Source language

```
type expr =  
  | Num of int  
  | Add of expr * expr  
  | Mul of expr * expr
```

Expressions contain integer constants, addition, and multiplication. Negative integer constants are allowed.

Target language

```
type inst =  
  | IPUSH of int  
  | IADD  
  | IMUL
```

```
type prog = inst list
```

- IPUSH *n* pushes *n* onto the stack.
- IADD pops two integers and pushes their sum.
- IMUL pops two integers and pushes their product.

2. Required Functions

```
val compute_expr : expr -> int  
val compile      : expr -> prog
```

`compute_expr` directly evaluates a source expression. `compile` emits code that leaves the expression's value on top of an initially empty stack. For a binary expression, compile the left operand, then the right operand, and finally emit the corresponding instruction.

```
val interp      : prog -> int  
val interp_opt  : prog -> int option  
  
exception StkMachineError of string  
val interp_exc  : prog -> int
```

`interp` executes a well-formed program and returns its single final value; behavior on ill-formed programs is intentionally unspecified. `interp_opt` and `interp_exc` provide explicit error handling.

Interpreter behavior

- `interp_opt` returns `Some n` when execution ends with exactly one value `n`.
- `interp_opt` returns `None` on stack underflow or if the final stack does not contain exactly one value.
- `interp_exc` returns the value on success and raises `StkMachineError` for the same error cases. The exception message is not prescribed.

3. Correctness

For every source expression `e`:

```
interp (compile e) = compute_expr e
```

The option- and exception-based interpreters must produce the same value on every compiled expression.

4. Implementation Requirements

- Use recursion and pattern matching; do not use mutable state or loops.
- Keep the supplied type definitions and required function signatures.
- Only `lib/compiler.ml` will be graded.
- Do not modify any other `.ml` files under `lib/`.
- You may freely modify `bin/main.ml` for your own testing; it will not be graded.
- The project must build with Dune.

5. Examples

```
let e = Add (Num 1, Mul (Num 2, Num 3))

compile e
= [IPUSH 1; IPUSH 2; IPUSH 3; IMUL; IADD]

compute_expr e = 7
interp (compile e) = 7
interp_opt (compile e) = Some 7
interp_exc (compile e) = 7

let bad = [IPUSH 1; IADD]

interp_opt bad = None
interp_exc bad raises StkMachineError
```

6. Tests

A small set of basic tests will be provided with the starter project. Your implementation should also handle nested expressions, negative integers, stack underflow, and final stacks containing zero or multiple values.